Group Meeting Minutes: StudentHaven Project (Polymorphism & Abstraction)

Meeting details

The group met on 10 November 2026 to agree on two object-oriented concepts for the StudentHaven project and add them to the code.

Item	Detail
Date	10 November 2026
Venue	Online platform
Project	StudentHaven, an off-campus student housing directory in Java
Present	Leonard K.M. Zimba 23126567 Widon Chirwa 23147068 Willard Kalaba Bob 24167830 Mwamba Mwaba 18123318

Agenda

1. Polymorphism
2. Abstraction: abstract methods and concrete methods
3. Adding both to the code

Discussion 1: Polymorphism

Polymorphism means one method call can behave differently depending on the object it runs on. The group agreed our User, Student and Admin classes are a good place to show it.

- Definition: a User reference can hold a Student or an Admin object, and Java runs that object's own version of an overridden method at run time.
- Type used: method overriding (run-time polymorphism), marked with @Override in the subclasses.
- Example from the code: displayMenu() is called the same way on both objects, but Student prints the student menu and Admin prints the admin menu.
- Example from the code: checkPassword() always fails for Student and compares the stored password for Admin.
- Benefit: a new user type could be added without changing the code that calls these methods.

Discussion 2: Abstraction, abstract methods and concrete methods

Abstraction means defining what every user must do in one template and leaving each user type to decide how. User is an abstract class, so it cannot be created directly; only Student and Admin objects can.

- Abstract method: declared with the abstract keyword and no body. Every subclass must implement it, or the code will not compile.
- Concrete method: has a full body in User and is shared by all subclasses as it is.

The group agreed on this split of methods in User:

Method	Kind	What it does
displayMenu()	Abstract	Shows the menu for that user type
getRole()	Abstract	Returns the role name, "Student" or "Administrator"
checkPassword(String)	Abstract	Decides whether a login attempt succeeds
canModifyListings()	Abstract	Says whether the user may add, edit or remove listings
getName()	Concrete	Returns the user's name, the same for everyone
showWelcome()	Concrete	Prints a welcome line built from getRole() and canModifyListings()

showWelcome() is concrete but relies on abstract methods, which ties abstraction back to polymorphism.

Decisions and code changes

The group agreed to add both concepts to StudentHaven.java without restructuring the existing code, and made these changes:

1. Kept displayMenu() abstract in User and made getRole(), checkPassword(String) and canModifyListings() abstract too.
2. Added the concrete method showWelcome() to User. It runs for Student at startup and for Admin after a successful login.
3. Made Student and Admin override every abstract method with their own behaviour.
4. Fixed a bug where a negative distance from campus was accepted: BoardingHouse now rejects it through setDistanceFromCampus(), and the admin input asks again until the value is 0 or more.
5. Compiled the program with no errors or warnings and tested both menus.

Action items and closing

- ☐ Every member reads the updated code and runs it once.
- ☐ Update the report and class diagram to show User as abstract.
- ☐ Prepare to explain polymorphism and abstraction in the demo.
- ☐ Decide whether editing a listing should change one field at a time.